Impact of Deep Compression Techniques on Phase-Specific Accuracy in TCN for Real-Time Gait Detection

Nontakorn Pulakkeaw
Computer Engineering
Mahidol University
Phra Nakhon Si Ayutthaya, Thailand
nonthakorn.dev@gmail.com

***Abstract*—Wearable-based gait phase detection is a critical enabler for rehabilitation monitoring, fall-risk assessment, and assistive robotics, yet deploying deep learning models on resource-constrained microcontrollers remains challenging. This study investigates the impact of deep compression techniques—post-training INT8 quantization and structured channel pruning—on the phase-specific accuracy of a Standard Temporal Convolutional Network (TCN) for real-time gait phase classification on the ESP32-C3 microcontroller. Using shank-mounted IMU data from the NONAN GaitPrint dataset, we evaluate all models under subject-wise stratified 5-fold cross-validation (K=5) with three random seeds (42, 123, 456). Validation subjects were re-assigned from the non-test pool independently for each fold. We systematically apply INT8 post-training quantization and structured channel pruning to TCN, quantify phase-specific accuracy degradation (LR, LS, PSw, Sw), and map the efficiency–accuracy trade-off via a Pareto front of macro F1-score versus on-device model size. Test evaluation uses stride-1 windows (highly overlapping); we report per-subject blocked macro F1 alongside window-level metrics. Under the primary K=5 × 3 baseline protocol, TCN achieved the highest mean macro F1 among compared architectures ($90.87\% \pm 1.47\%$), significantly outperforming CNN1D ($+0.53$ pp, Wilcoxon p=0.004) while remaining statistically equivalent to Transformer (p=0.80) and LSTM+GRU (p=0.17). K-fold compression (N=15 matched fold×seed pairs per config) shows that INT8 quantization is effectively lossless ($90.88\% \pm 1.47\%$ vs. FP32; p=0.30). Structured pruning at 50% channel retention with a 5-epoch fine-tune budget reduces macro F1 by 1.47 pp (p<0.001); extending the fine-tune schedule to 50 epochs recovers 0.94 pp ($90.34\% \pm 1.11\%$, p=0.015 vs. FP32), indicating that part of the pruning loss is attributable to insufficient recovery training. PSw F1—the most compression-sensitive transition phase—drops by up to 3.75 pp under aggressive Prune25 (p<0.001). INT8+Prune50 yields the smallest ESP32-deployable footprint (∼5.3 KB) at a 1.54 pp macro F1 cost ($p=1.2\times10^{-4}$). On-device ESP32-C3 latency measurements are pending TFLite re-export.**

***Index Terms*—TCN, deep compression, quantization, pruning, gait phase detection, ESP32-C3, edge AI, wearable sensors**

I. Introduction

Accurate detection of gait phases—such as loading response, mid-stance, pre-swing, and swing—is fundamental to applications in clinical rehabilitation, fall prevention, prosthetic control, and sports biomechanics. Recent advances in deep learning, particularly recurrent and convolutional architectures, have substantially improved gait phase classification accuracy using wearable inertial measurement unit (IMU) sensors. However, these models are typically designed and validated on desktop or server-class hardware, and their large parameter counts and floating-point computations make them impractical for direct deployment on low-power, low-memory microcontrollers used in real-world wearable devices.

Temporal Convolutional Networks (TCNs), which leverage dilated convolutions to capture long-range temporal dependencies with fewer parameters than recurrent architectures, offer a promising middle ground between accuracy and efficiency and can be further compressed via quantization and pruning to meet the strict memory and latency budgets of microcontroller-class hardware such as the ESP32-C3.

While prior work has explored model compression broadly for time-series classification, relatively little attention has been paid to how compression affects accuracy unevenly across different gait phases. Transition phases such as Loading Response (LR) and Pre-Swing (PSw) are inherently more ambiguous due to overlapping biomechanical signal characteristics, and we hypothesize that these phases are more vulnerable to the information loss introduced by aggressive quantization and pruning than steady-state phases such as Swing (Sw).

This study addresses this gap by:

1) Selecting and training a Standard TCN backbone for gait phase classification using shank IMU data from the NONAN GaitPrint dataset, with matched-capacity FP32 baselines for context;
2) Systematically applying INT8 post-training quantization, as well as structured channel pruning, and quantifying their impact on phase-specific accuracy and F1-score, rather than overall accuracy alone;
3) Deploying the compressed models on ESP32-C3 hardware and measuring on-device inference latency and SRAM headroom.

The remainder of this paper is organized as follows: Section II reviews related work on gait phase detection, TCN architectures, and edge AI compression. Section III describes the methodology, including dataset preparation, model architecture, and compression protocols. Section IV presents

experimental results. Section V discusses the implications of phase-specific degradation and hardware feasibility, and Section VI concludes the paper.

II. Literature Review

A. Gait Phase Detection Using Wearable IMU Sensors

Gait phase detection using body-worn IMU sensors has been extensively studied, with shank- and foot-mounted sensors commonly used due to their high sensitivity to phase transitions. Classical machine learning approaches (e.g., threshold-based, hidden Markov models) have largely been superseded by deep learning methods, including 1D Convolutional Neural Networks (CNNs), Long Short-Term Memory (LSTM) networks, hybrid LSTM+GRU architectures, and—more recently—Transformer encoders, which achieve higher accuracy by learning temporal dependencies directly from raw or lightly processed sensor signals. Using the same NONAN GaitPrint dataset and shank-mounted, six-axis IMU configuration adopted in this study, Choi and Choi [1] directly compared a 1D CNN, a hybrid LSTM+GRU model, and a Transformer for four-class gait phase classification, reporting comparable performance across the three architectures (macro F1 approximately 91–93%) with the Transformer attaining the highest score. This paper adopts the same three architectures, plus a standard TCN, as FP32 baselines under an identical protocol (Section III), but targets a distinct research question left open by prior work: how INT8 quantization and structured pruning affect *phase-specific* accuracy, and how the resulting models perform when deployed on microcontroller-class hardware.

B. Temporal Convolutional Networks (TCNs) for Time-Series Classification

TCNs use causal, dilated convolutions to achieve large receptive fields with a fraction of the parameters required by recurrent architectures, while supporting parallelized computation [2]. This property makes TCNs particularly attractive for edge deployment, where both parameter count and inference latency are critical constraints. In this study, all architectures share a common hidden width (32 channels) and 0.5 s causal windows, yielding 10–26K-parameter models sized for ESP32-C3 deployment rather than server-scale capacity.

C. Model Compression Techniques: Quantization and Pruning

Post-training quantization reduces numerical precision (e.g., from 32-bit floating point to 8-bit integer representations), substantially reducing model size and enabling faster, more energy-efficient inference on integer-optimized microcontroller hardware [3]. Structured pruning removes entire channels or filters, yielding actual speedups on commodity hardware (unlike unstructured pruning, which often requires specialized sparse-matrix support) [4]. Both techniques have been widely studied for image and audio models, but their differential effect across distinct temporal phases of a periodic signal—such as gait phases—remains comparatively underexplored.

D. Edge AI Deployment on Microcontrollers (ESP32-C3)

TensorFlow Lite Micro (TFLM) enables compact neural networks to run directly on microcontroller hardware with kilobyte-scale memory budgets [5]. The ESP32-C3 is a single-core RISC-V microcontroller (160 MHz) with approximately 400 KB of SRAM [6], making it a low-cost target for TinyML deployment. Although it lacks the vector acceleration available on higher-end Espressif parts, compact INT8 models can still run on-device when combined with structured pruning, motivating its use as the deployment platform in this study.

E. Research Gap

While compression-accuracy trade-offs are well documented at the aggregate level, the literature provides limited insight into which specific gait phases are most degraded by compression. This is a meaningful gap because transition phases (LR, PSw) are often clinically and functionally the most informative (e.g., for fall-risk detection), yet may also be the most fragile under information loss. This paper directly addresses this gap.

III. Methodology

A. Dataset and Preprocessing

- **Dataset:** NONAN GaitPrint [7], using shank-mounted IMU channels (accelerometer and gyroscope axes).
- **Sampling rate:** Original 200 Hz signal downsampled to 100 Hz for all reported experiments, balancing temporal resolution against on-device resource constraints.
- **Labeling:** Gait cycle phases labeled as Loading Response (LR), Loading Stance (LS), Pre-Swing (PSw), and Swing (Sw).
- **Data split:** Subject-wise stratified 5-fold CV (K=5) with three seeds; validation subjects rotate per fold (resampled from the non-test pool) to avoid repeated early-stopping bias; no subject leakage across train/val/test.
- **Normalization:** Per-channel z-score normalization computed on the training set only.
- **Windowing:** Causal 0.5 s windows with train stride 5 and validation/test stride 1 (test windows overlap ≈98%; per-subject blocked macro F1 reported alongside window-level metrics).
- **Training:** Adam optimizer ($\mathrm{lr} = 10^{-3}$), batch size 512, up to 50 epochs; best checkpoint selected by validation macro F1.
- **Fair comparison design:** All architectures use hidden width 32, four-class output, and identical windowing, optimization, and cross-validation splits. Model capacity is intentionally capped ($\sim$10–26K parameters) for microcontroller deployment ($\sim$40–100 KB FP32) rather than maximizing accuracy with large networks.
- **Compression fine-tuning:** Pruning uses *training subjects only* (fixed epoch budget; no validation access) to avoid double-dipping on the validation set used for FP32 checkpoint selection. Runtime assertions verify zero subject overlap between training/fine-tuning and held-out test partitions for every fold.

- **Evaluation:** Table II reports mean $\pm$ std macro F1 over $K{=}5$ folds$\times$3 seeds ($N{=}15$ runs per model) with paired Wilcoxon tests. Primary compression results use the same k-fold protocol (Table III, Wilcoxon vs. FP32 on $N{=}15$ pairs). Tables IV and V report the extended Pareto grid and fine-tune schedule ablation on the same splits. Figs. 2 and 3 provide a qualitative single-run illustration (fold 0, seed 42). ESP32 latency (Table VI) is pending TCN TFLite re-export.

Phase support is highly imbalanced (LS and Sw together account for $(1{,}274{,}499 + 1{,}016{,}857)/2{,}583{,}680 \approx 88.7\%$ of test windows, vs. only 11.3% for the transition phases LR and PSw combined). To prevent the majority phases from dominating optimization, all models—including the four FP32 baselines and every compression fine-tuning stage—are trained with a class-balanced weighted cross-entropy loss:

$$\mathcal{L} = -\sum_{c=1}^{4} \alpha_c \, y_c \log \hat{y}_c, \qquad \alpha_c = \frac{N}{4 \cdot N_c}, \tag{1}$$

where N is the number of training windows, N_c is the number of windows labeled with phase c, y_c is the one-hot ground-truth indicator, and $\hat{y}_c$ is the softmax output probability for phase c. This directly motivates reporting *macro*, rather than micro-averaged, metrics throughout this paper.

B. TCN Architecture

We adopt a Standard TCN [2] as the deployment backbone. The model consists of four dilated causal convolutional blocks (dilation rates 1, 2, 4, and 8) with residual connections, batch normalization, and a lightweight classification head (global average pooling followed by a dense softmax layer over the four phase classes). Each block computes a dilated causal convolution (kernel size $k{=}3$, dilation d, ReLU) so that each output depends only on present and past inputs. Stacking four such blocks yields a receptive field of 31 samples (0.31 s at 100 Hz), comfortably within the 0.5 s (50-sample) input window while keeping the parameter count near 11K. The model consumes causal 0.5-second windows (50 samples at 100 Hz). Channel width is deliberately kept narrow (32 hidden channels) to retain a microcontroller-deployable footprint ($\sim$45 KB FP32). Fig. 1 illustrates the overall data flow and the internal structure of each causal convolutional block.

C. Baseline Architectures for Context

The primary comparison in this work is across compression configurations of TCN (FP32 vs. INT8 vs. pruned). For context, three FP32 baselines (CNN1D, LSTM+GRU, Transformer) were retrained under the same hidden width, windowing, and cross-validation protocol (Table II). CNN1D uses three Conv1D layers; LSTM+GRU stacks single-layer LSTM and GRU encoders; the Transformer uses two encoder layers with four heads ($d{=}32$).

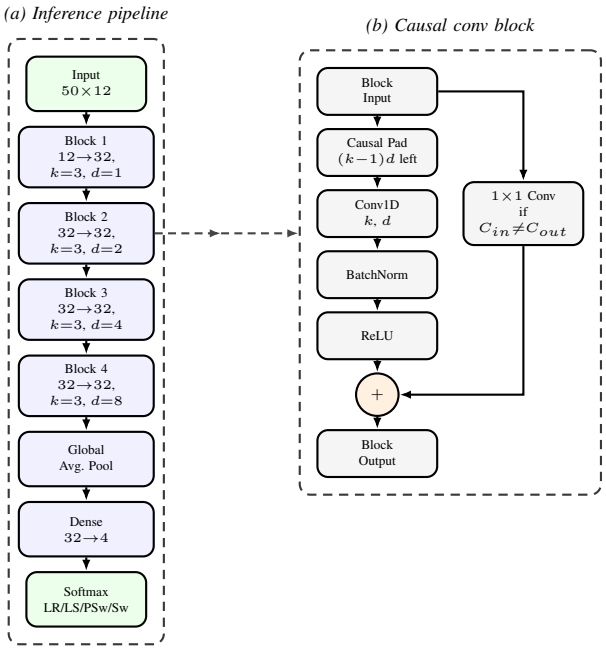

Fig. 1. Standard TCN architecture: (a) end-to-end pipeline from a $50{\times}12$ causal window to a 4-class softmax, and (b) internal structure of each causal convolutional block (left-padding, dilated Conv1D, batch norm, ReLU, and residual connection).

D. Deep Compression Protocol

Post-training quantization (INT8). We apply symmetric 8-bit quantization separately to each convolutional and fully connected weight layer. For every layer, a single scale is set from that layer's largest absolute weight; each weight is rounded to the nearest integer in $[-127, 127]$, then mapped back to floating point for inference. This quantize–dequantize (QDQ) proxy is used for PyTorch-side test-set evaluation (Section IV). For ESP32 deployment, models are exported as full-integer TensorFlow Lite graphs using TFLite's built-in post-training quantization.

Structured channel pruning. The TCN stem uses 32 hidden channels. Channels are ranked by summed absolute weight magnitude across all stem convolutional blocks; the top-k channels are retained and removed elsewhere in the stem and classification head. The pruned model is fine-tuned on *training subjects only* (Adam, learning rate 10^{-4}, class-balanced loss) before re-evaluation. Primary k-fold configs sweep 25%, 50%, and 75% channel retention (Prune25/50/75) with a default 5-epoch fine-tune budget; Tables V and IV report the same protocol at $N{=}15$.

Extended compression grid (k-fold). Beyond FP32/INT8 and Prune25/50/75, combined INT8+Prune50 populates the Pareto front (Table IV). INT4 is excluded because full-integer INT4 TFLite is not supported on the ESP32-C3 deployment target. Fine-tune schedule ablation on Prune50 (5/15/50 epochs) isolates whether accuracy loss under pruning reflects insufficient recovery training rather than an inherent pruning limit (Table V).

Table I summarizes the compression techniques applied to

TABLE I
DEEP COMPRESSION TECHNIQUES APPLIED TO TCN

Technique	Variant	Description
Quantization	INT8 (PTQ)	PyTorch QDQ proxy for test metrics; full-integer TFLite PTQ for ESP32
Pruning	Structured 25/50/75%	Drop least-salient hidden channels; default 5-epoch fine-tune on train only
Combined	INT8+Prune50	Prune then quantize for the smallest ESP32-deployable footprint

TCN.

E. Hardware Deployment (ESP32-C3)

After PyTorch compression evaluation, selected models are converted to TensorFlow Lite (.tflite) format and deployed via TensorFlow Lite Micro on an ESP32-C3 development board (160 MHz, Arduino IDE, Chirale_TensorFlowLite). On-device inference latency is measured per 0.5-second input window over 1000 timed trials (20 warmup invocations), alongside minimum free heap during benchmarking and a fixed 120 KB tensor arena.

F. Evaluation Metrics

Macro F1 is the unweighted mean of the four per-phase F1 scores. Unlike accuracy or a support-weighted average, this gives Loading Response (LR) and Pre-Swing (PSw) equal weight to the majority phases despite their $\sim 6\%$ and $\sim 5\%$ test-set support, respectively.

- **Phase-specific F1-score and accuracy** for LR, LS, PSw, and Sw individually.
- **Pareto analysis:** macro F1-score vs. measured or estimated model size (KB) per compression config.
- **Model complexity:** parameter count and payload size.
- **On-device performance:** inference latency (ms) and minimum free heap (KB) on ESP32-C3.
- **Efficiency-accuracy trade-off** visualized via a Pareto front across compression configurations.

IV. RESULTS

A. Baseline Comparison and Model Complexity

Table II compares TCN against three FP32 baselines under the primary $K{=}5 \times 3$ protocol (mean $\pm$ std over 15 runs). Macro F1 scores cluster tightly (90.34–90.87%); TCN attains the highest mean macro F1 ($90.87\% \pm 1.47\%$). Paired Wilcoxon tests on matched fold$\times$seed pairs show TCN statistically equivalent to Transformer ($\Delta{=}+0.04$ pp, $p{=}0.80$) and LSTM+GRU ($+0.28$ pp, $p{=}0.17$), but significantly higher than CNN1D ($+0.53$ pp, $p{=}0.004$; paired t-test $p{=}0.005$). Fold 1 is the hardest stratum for all models ($\sim$88–89% macro F1); $\sigma_{\text{fold}} \gg \sigma_{\text{seed}}$ for TCN (1.38 vs. 0.28 pp), indicating subject heterogeneity dominates seed noise.

TABLE II
BASELINE FP32 COMPARISON ($K{=}5 \times 3$ SEEDS)

	TCN	Trans.	LSTM	CNN1D
Params	11,560	25,956	12,356	10,727
Size (KB)	45.2	101.4	48.3	41.9
Macro F1	**90.87**	90.83	90.59	90.34
	$\pm$1.47	$\pm$0.99	$\pm$1.19	$\pm$1.21
Phase F1 (%)				
LR	**85.81**	85.37	84.95	84.32
LS	**97.78**	97.73	97.70	97.52
PSw	82.72	82.95	82.52	82.54
Sw	97.18	**97.27**	97.22	97.00

Mean $\pm$ std ($N{=}15$). All models: hidden width 32 (Sec. III-B).

TABLE III
TCN K-FOLD COMPRESSION (MACRO F1 %, MEAN $\pm$ STD; $N{=}15$ PER CONFIG)

Config	Macro F1	PSw F1	Wilcoxon p vs. FP32
FP32	90.87 ± 1.47	82.72 ± 2.54	—
INT8	90.88 ± 1.47	82.75 ± 2.54	0.303
Prune25	87.62 ± 1.45	78.97 ± 1.28	6.10e-05
Prune50	89.40 ± 1.43	81.13 ± 1.97	6.10e-05
Prune75	90.17 ± 1.30	81.90 ± 2.18	6.10e-05
INT8+Prune50	89.33 ± 1.36	80.93 ± 2.00	1.22e-04

Paired Wilcoxon signed-rank tests on matched fold$\times$seed pairs ($N{=}15$) vs. FP32. All configs complete (120/120 jobs).

B. TCN Compression Under K-Fold Cross-Validation

Table III reports the primary compression results aggregated over $N{=}15$ matched fold$\times$seed pairs, with paired Wilcoxon signed-rank tests against the FP32 reference (same checkpoints, no retraining).

INT8 quantization is statistically indistinguishable from FP32. INT8 attains $90.88\% \pm 1.47\%$ macro F1 ($+0.01$ pp vs. FP32; Wilcoxon $p{=}0.30$) with an estimated payload of $\sim$13.0 KB ($\sim 3.4\times$ smaller than FP32). Per-phase F1 changes are at most 0.03 pp (PSw: 82.75% vs. 82.72%), confirming that 8-bit weight precision is sufficient for this TCN under the PyTorch QDQ proxy.

Pruning severity trades size for accuracy in a monotonic pattern. Prune25 ($\sim$4.3 KB) reduces macro F1 by 3.25 pp ($87.62\% \pm 1.45\%$, $p{<}0.001$), with the largest phase-specific loss in LR (-6.98 pp) and PSw (-3.75 pp). Prune50 ($\sim$13.1 KB) costs 1.47 pp ($89.40\% \pm 1.43\%$, $p{<}0.001$). Prune75 ($\sim$26.4 KB) costs 0.70 pp ($90.17\% \pm 1.30\%$, $p{<}0.001$). All pruned configs remain significantly below FP32 at $\alpha{=}0.05$, but the effect size shrinks as more channels are retained.

Combined INT8+Prune50 minimizes size at a measurable accuracy cost. INT8+Prune50 ($\sim$5.3 KB, smallest ESP32-deployable point) achieves $89.33\% \pm 1.36\%$ macro F1 (-1.54 pp, $p{=}1.2{\times}10^{-4}$), with PSw F1 falling to $80.93\% \pm 2.00\%$ (-1.79 pp).

Table IV summarizes the extended grid with parameter counts and payload sizes. Among non-dominated configurations, INT8 offers the best accuracy–size trade-off;

TABLE IV
EXTENDED COMPRESSION GRID (K-FOLD; SIZE FROM FIRST COMPLETED RUN PER CONFIG)

Config	Params	KB	Macro F1	PSw F1
INT8	11,300	13.0	90.88 ± 1.47	82.75 ± 2.54
Prune25	1,100	4.3	87.62 ± 1.45	78.97 ± 1.28
Prune50	3,348	13.1	89.40 ± 1.43	81.13 ± 1.97
Prune75	6,748	26.4	90.17 ± 1.30	81.90 ± 2.18
INT8+Prune50	3,348	5.3	89.33 ± 1.36	80.93 ± 2.00

TABLE V
PRUNE50 FINE-TUNING SCHEDULE ABLATION (K-FOLD; N=15; WILCOXON VS. FP32)

Config	FT ep.	Macro F1	Δ vs. FP32	PSw F1	p
Prune50	5	89.40 ± 1.43	-1.47	81.13 ± 1.97	6.10e-05
Prune50 (15 ep)	15	89.83 ± 1.54	-1.04	81.29 ± 2.79	6.10e-04
Prune50 (50 ep)	50	90.34 ± 1.11	-0.53	82.23 ± 1.82	0.015

INT8+Prune50 targets minimum memory when a ∼1.5 pp macro F1 reduction is acceptable.

C. Fine-Tuning Schedule Ablation

Table V isolates whether accuracy loss under Prune50 reflects an insufficient post-prune recovery budget rather than an inherent pruning limit. With the default 5-epoch fine-tune, Prune50 macro F1 is 1.47 pp below FP32 (p<0.001). Extending fine-tuning to 15 epochs recovers 0.43 pp ($89.83\% \pm 1.54\%$, p=6.1×10^{-4} vs. FP32), and 50 epochs recovers 0.94 pp ($90.34\% \pm 1.11\%$, p=0.015 vs. FP32). PSw F1 improves from $81.13\% \pm 1.97\%$ (5 ep) to $82.23\% \pm 1.82\%$ (50 ep), approaching the FP32 baseline ($82.72\% \pm 2.54\%$). Residual macro F1 gap at 50 epochs (−0.53 pp, still significant at α=0.05) indicates that structured 50% channel removal retains a small but reproducible information loss beyond what longer fine-tuning can recover.

D. Compression Trade-offs and Phase Degradation

Figs. 2 and 3 qualitatively illustrate the accuracy–size trade-off and phase-specific degradation patterns on a single fold×seed pair (fold 0, seed 42). These figures are consistent with the k-fold aggregates in Tables III–V: INT8 is near-lossless, structured pruning disproportionately affects transition phases (LR, PSw), and INT8+Prune50 occupies the smallest footprint at the largest accuracy cost.

Fig. 3 complements Tables III and V by visualizing per-phase *accuracy* rather than F1, highlighting that accuracy alone can mask precision loss on minority transition phases.

E. On-Device Performance on ESP32-C3

On-device latency and SRAM measurements require exporting each compressed TCN checkpoint to TFLite and re-running the ESP32-C3 benchmark pipeline. Table VI is reserved for these results; prior measurements from an earlier export pipeline are not reported here because the deployment backbone changed to Standard TCN.

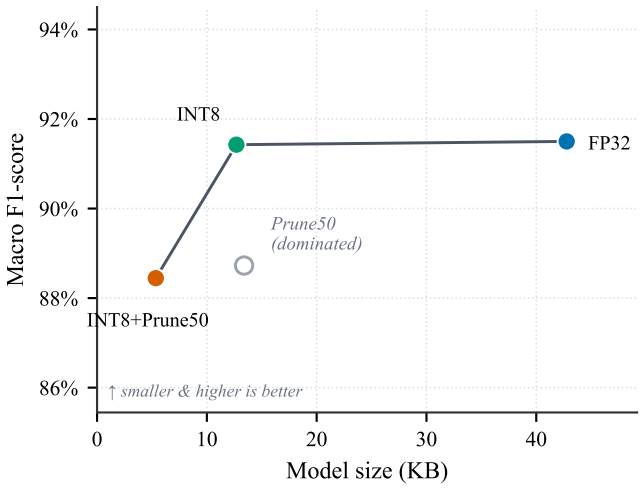

Fig. 2. Illustrative Pareto front (fold 0, seed 42; N=1). K-fold aggregates with Wilcoxon tests appear in Tables III and IV. INT8 (filled marker) dominates Prune50 on both size and macro F1 in this run.

TABLE VI
ON-DEVICE INFERENCE ON ESP32-C3 (TCN EXPORT PENDING)

Config	Mean (ms)	p95 (ms)	TFLite (KB)	Heap (KB)
FP32	—	—	—	—
INT8	—	—	—	—
Prune50	—	—	—	—
INT8+Prune50	—	—	—	—

Pending TCN TFLite export and ESP32-C3 re-measurement (160 MHz; arena 120 KB; 1000 trials).

V. DISCUSSION

A. Variance Across Folds and Seeds

Track B aggregates 15 runs per model (K=5 folds×3 seeds). For TCN, σ_{fold}=1.38 pp exceeds σ_{seed}=0.28 pp, so performance spread primarily reflects which subjects appear in each test fold rather than initialization noise. Paired Wilcoxon tests on matched fold×seed pairs show TCN statistically equivalent to Transformer (Δ=+0.04 pp, p=0.80) and LSTM+GRU (+0.28 pp, p=0.17), but significantly higher than CNN1D (+0.53 pp, p=0.004).

B. Vulnerability of Transition Phases

Across k-fold compression (Table III), PSw F1 shows the largest pruning-related degradation relative to FP32: −3.75 pp (Prune25), −1.59 pp (Prune50, 5 ep), and −1.79 pp (INT8+Prune50), whereas INT8 quantization changes PSw F1 by only +0.03 pp (p=0.30 vs. FP32). LR F1 is equally sensitive under aggressive pruning (−6.98 pp for Prune25). The fine-tune ablation (Table V) shows that extended recovery training partially restores PSw F1 ($81.13\% \rightarrow 82.23\%$ from 5 to 50 epochs) but does not fully close the gap to FP32 (82.72%). PSw is a short transition window whose decision boundary likely depends on finer temporal detail removed when hidden channels are pruned.

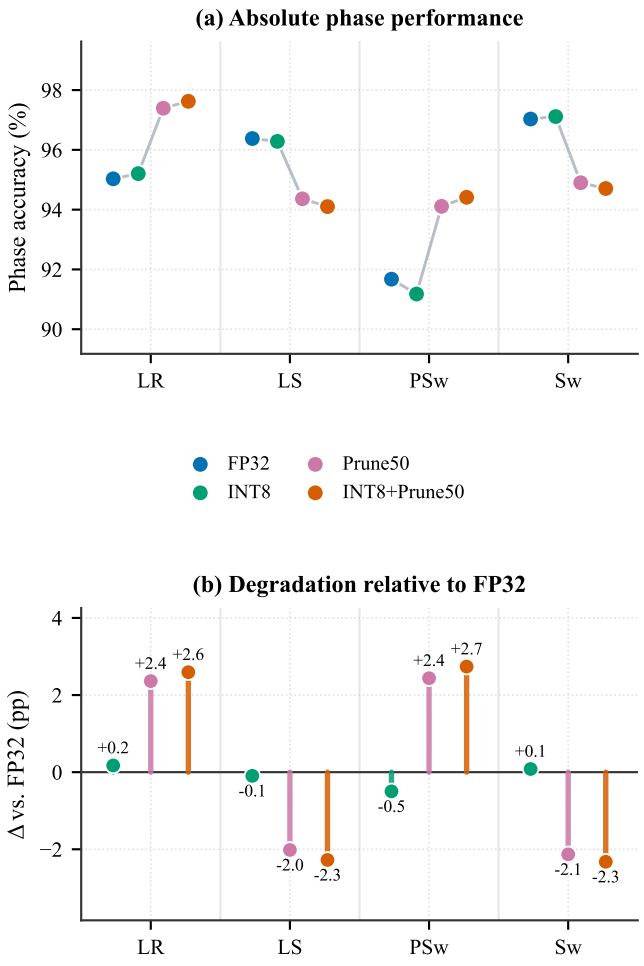

Fig. 3. Phase-specific test *accuracy* under compression (fold 0, seed 42 illustration). Panel (b) shows that LR accuracy can increase under pruning even when LR F1 falls (Table III), motivating macro and phase F1 over accuracy alone.

C. Role of Dilated Convolutions Under Compression

INT8 quantization preserved macro F1 within 0.01 pp of FP32 across $N{=}15$ k-fold runs (Wilcoxon $p{=}0.30$), suggesting that TCN's dilated receptive field and narrow hidden width (32 channels) remain sufficient under 8-bit weight precision. Structured pruning removes representational capacity: even with 50-epoch fine-tuning, Prune50 remains 0.53 pp below FP32 ($p{=}0.015$), and PSw—already the lowest baseline phase (82.72% F1)—is disproportionately affected.

D. Efficiency-Accuracy Trade-off

The k-fold Pareto analysis (Tables III and IV) identifies INT8 ($\sim$13.0 KB, $+0.01$ pp macro F1, $p{=}0.30$) as the preferred near-lossless compression point for ESP32-C3 deployment. INT8+Prune50 ($\sim$5.3 KB) trades 1.54 pp macro F1 ($p{=}1.2{\times}10^{-4}$) for minimum memory. Prune75 offers a middle ground ($\sim$26.4 KB, -0.70 pp) when pruning without quantization is desired. Deployment latency must be paired with these accuracy/size trade-offs; Table VI remains pending TCN TFLite export and ESP32-C3 re-measurement.

E. Feasibility for Real-World Wearable Deployment

Once Table VI is populated, latency can be compared against the intended inference schedule (e.g., every 50–100 ms at 100 Hz with stride 5–10). Host-side TFLite smoke validation will be re-run after TCN export to confirm argmax agreement between PyTorch and on-device graphs.

F. Limitations

- Results are derived from a single public dataset (NONAN GaitPrint); generalization to other populations, walking speeds, or pathological gait patterns requires further validation.
- ESP32-C3 on-device latency and SRAM headroom (Table VI) were pending at the time of writing; PyTorch-side conclusions await TFLite export confirmation.
- Test stride 1 yields highly overlapping windows; per-subject blocked aggregation mitigates inflated effective sample size, but Wilcoxon tests operate on fold$\times$seed aggregates ($N{=}15$) rather than subject-blocked scores.
- On-device benchmarks will use a synthetic replay window, not live IMU streaming; end-to-end latency with BLE acquisition remains unmeasured.
- PyTorch INT8 evaluation uses a quantize-dequantize weight proxy; deployed INT8 TFLite models use full-integer PTQ and may differ slightly in logits while preserving argmax in our smoke tests.
- Power consumption (mW) was not measured on ESP32-C3.
- Latency will use TFLite Micro reference kernels (Chirale_TensorFlowLite); optimized INT8 kernels could reduce inference time further.

VI. Conclusion

This study systematically evaluated how INT8 quantization and structured channel pruning affect phase-specific accuracy in a Standard TCN for gait phase detection. Under $K{=}5 \times 3$ cross-validation, TCN achieved $90.87\% \pm 1.47\%$ macro F1—the highest mean among compared architectures while remaining microcontroller-deployable. K-fold compression with paired Wilcoxon tests ($N{=}15$ per config) showed that INT8 is statistically equivalent to FP32 ($p{=}0.30$), whereas structured pruning incurs significant but severity-dependent losses (Prune50 at 5 fine-tune epochs: -1.47 pp, $p{<}0.001$; Prune75: -0.70 pp, $p{<}0.001$). Fine-tune schedule ablation revealed that 0.94 pp of the Prune50 deficit is recoverable with longer training (50 epochs: $90.34\%{\pm}1.11\%$, $p{=}0.015$ vs. FP32), implicating insufficient recovery budget—not pruning alone—as a contributor to early results; a residual 0.53 pp gap nonetheless persists. PSw F1 was the most compression-sensitive transition phase. INT8+Prune50 ($\sim$5.3 KB) offers minimum memory at a 1.54 pp cost ($p{=}1.2{\times}10^{-4}$). Future work will export TCN to TFLite, measure ESP32-C3 latency and power, and validate live IMU streaming.

ACKNOWLEDGMENT

The authors thank the creators of the NONAN GaitPrint dataset for making their IMU gait recordings publicly available, which made this study possible.

REFERENCES

[1] W. Choi and M.-T. Choi, "Deep learning-based gait phase detection using shank-mounted IMU data: Classification approach," *PLOS ONE*, vol. 21, no. 4, p. e0344002, 2026, doi: 10.1371/journal.pone.0344002.

[2] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018.

[3] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[4] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.

[5] R. David et al., "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proc. Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 800–811.

[6] Espressif Systems, *ESP32-C3 Technical Reference Manual*, version 1.3, 2024. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-c3_technical_reference_manual_en.pdf

[7] T. M. Wiles et al., "NONAN GaitPrint: An IMU gait database of healthy young adults," *Scientific Data*, vol. 10, no. 867, 2023, doi: 10.1038/s41597-023-02704-z.